EcoGuard Team Split & Crossovers

Oct 25, 2026 · @Dylan

Rubric ownership

Member	Backend rubric items (marks)	UI rubric items (marks)
1. Identity and cross-cloud	1.1 Sign-up, sign-in, sign-out (4); 1.2 Access control and redirection (3); 1.3 Fine-grained permissions and multi-cloud auth (3); 2.3.3 Tag-based notifications (4)	3.1 Sign-up and sign-in screens, shared with 2 (5 total); 3.4 Overall usability through the app shell and routing, shared with 2 and 4 (5 total); notification settings page supports the 2.3.3 demo
2. Ingestion and storage (you)	2.1.1 File upload and deduplication (5); 2.1.2 Thumbnails and video processing (5); 2.3.2 File and record deletion (3); Firestore schema that 2.1.3 and every query depend on	3.1 Upload UI with progress and duplicate feedback, shared with 1; 3.4 File deletion controls, shared with 1 and 4
3. ML and detection	2.1.3 ML tagging, extensibility and DB insertion (6); 2.2.3 Find by uploaded file (5); 2.2.4 Lost pet search (5)	3.2 Uploaded-file search and lost pet search screens, shared with 4 (5 total)
4. Search and chatbot	2.2.1 Find by tags (5); 2.2.2 Find by thumbnail URL (3); 2.3.1 Manual bulk tag editing (3); 2.2.5 Lex chatbot (6)	3.2 Tag and count query UI and results gallery, shared with 3; 3.3 Lex chatbot UI (5); 3.4 Bulk tag editing controls, shared with 1 and 2
Everyone	Multi-cloud communication shown and justified in the demo	None

Crossovers and conflicts

You (member 2) sit in 12 of the 17 crossovers, most of them with member 3, because the Firestore record, upload triggers and video frames pass straight from your functions into theirs. High means it breaks integration or costs a rubric item if left until the end.

Members	Shared piece	Risk	What could go wrong	Agree up front
2, 3	Firestore file record	High	Field names or tag shape differ; the upload function overwrites tags written by tagging, or tagging writes before the record exists	Member 2 commits the schema to the repo first; document ID is the SHA-256; each function updates only its own fields with a merge, never a full overwrite
2, 3	Upload event triggers	High	Thumbnail and tagging functions fire on the same upload in either order; tagging needs the thumbnail URL; thumbnail writes re-trigger functions in a loop	Thumbnail key derived from the file ID, so tagging can record it without waiting; thumbnails and frames go to prefixes or buckets the triggers ignore
2, 3	Video frames	High	No agreed handoff for frames, frames never cleaned up, or counts summed across frames	Member 2 extracts 1 frame per 10 s and hands frames to member 3′s detector one agreed way (in memory or a temporary prefix); counts use the per-frame maximum; frames deleted after tagging
3, 4	Tag vocabulary	High	Model labels (Pig, Fox) never match query terms or Lex slot values (feral pig, fox), so searches return nothing	One lowercase tag list in a shared config file; member 3 maps model labels onto it; member 4′s Lex slot type uses the same list

Members	Shared piece	Risk	What could go wrong	Agree up front
1, 2, 3, 4	GCP API Gateway and Cognito tokens	High	Three people add routes to one gateway spec and conflict; an endpoint left off the gateway stays public (1.2 drops to 50%)	Member 1 owns the gateway spec and merges route requests; the frontend sends the token through one shared API client; test every endpoint without a token
1, 4	Lex bot reaching GCP	High	The fulfilment Lambda calls GCP with no valid token, or a hardcoded key (1.3 drops to 0%); the browser widget cannot call Lex without AWS credentials	Decide whether the Lambda forwards the user's Cognito token or uses member 1's federation; the widget gets credentials from a Cognito identity pool set up by member 1
All	Cloud accounts and credits	High	Each member deploys to their own account, so nothing connects at integration; AWS Academy lab accounts cannot add teammates	One shared AWS account and one GCP project with every member added; agree whose credits pay for what
2, 4	Thumbnail naming	Medium	Member 4's thumbnail lookup (2.2.2) guesses the original's name and fails for PNG or WebP originals	Thumbnails named `thumbs/<sha256>.jpg`; original key and extension stored in Firestore

Members	Shared piece	Risk	What could go wrong	Agree up front
2, 4	URLs as API input	Medium	Delete (2.3.2) and bulk tagging (2.3.1) both take URL lists but parse presigned or encoded URLs differently, so one works and the other returns 404	One shared URL-to-file-ID helper; store bucket and key, and sign URLs only in responses
2, 3	Query photos and re-uploads	Medium	Query-by-file or lost pet photos pass through the upload pipeline and get stored and indexed (2.2.3 drops to 50%); a duplicate upload re-runs tagging and wipes manual tags	Query photos go to member 3′s endpoint in memory, never to the media bucket; the dedup check happens before a signed URL is issued, so duplicates never fire a trigger
3, 4	Manual and automatic tags	Medium	Re-tagging after a model update overwrites tags added by hand; manual tags have no agreed count	Manual add sets the count to 1 if the tag is absent; removal deletes the key; re-tagging keeps tags flagged as manual
3, 4	Shared query logic	Medium	Query-by-file and lost pet search reimplement tag matching with different AND or count rules; the chatbot′s lost pet intent has nothing to call	Member 3 calls member 4′s tag query after detection; member 4′s Lex fulfilment calls member 3′s lost pet endpoint; request and response JSON written down

Members	Shared piece	Risk	What could go wrong	Agree up front
1, 3, 4	Notification events	Medium	Nobody publishes the event member 1 listens for, or every Firestore write (thumbnail URL, dedup) sends an email	Member 1 triggers on Firestore writes and alerts only when the tags map changes, which covers tagging and manual edits with no code from members 3 or 4
All	Least-privilege roles	Medium	Functions share one broad service account or wildcard role; member 1 is marked on 1.3 but cannot fix it at the end	Each owner writes a role limited to what their function touches; member 1 reviews them all before the demo
1, all	App shell and routing	Medium	Pages built in different frameworks or styles don't fit the shell; everyone edits the same route and navigation files	Framework, styling and a folder per page agreed now; member 1 adds routes on request
2, 3, 4	Results gallery component	Medium	Three galleries get built; selection for delete or tagging works in one but not another; thumbnails on member 3's pages don't expand to full size (3.2 drops to 50%)	Member 4 builds one gallery with selection and full-size expand; members 2 and 3 plug their actions and results into it

Members	Shared piece	Risk	What could go wrong	Agree up front
All	Report, diagram and Git history	Low	Unowned report sections get written the night before; one person merges others' code, so commit history looks unbalanced (4.2.1)	Assign report owners now; everyone commits their own work through their own branch and pull request

Cloud services by requirement

Put identity, the chatbot and email on AWS, using the three services the spec names (Cognito, Lex, SNS), and keep files, data and compute together on GCP. Files then never cross clouds, and the demo still shows three real cross-cloud links (diagram below).

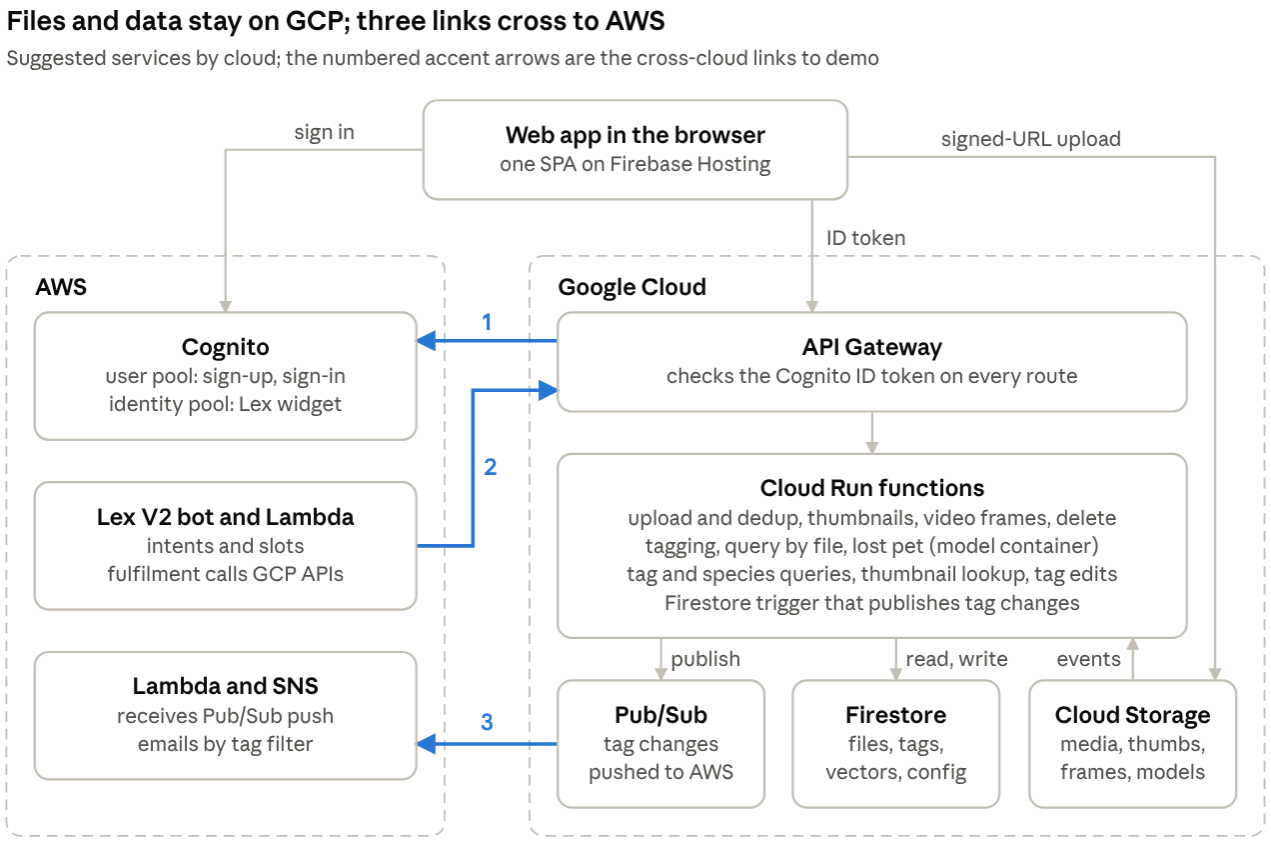
Rubric item	Owner	Cloud	Services	Why, and the alternative
1.1 Sign-up, sign-in, sign-out	1	AWS	Cognito user pool with managed login (formerly Hosted UI), or custom pages calling Cognito through Amplify Auth	The spec requires Cognito; managed login gives the verification and password flows with no UI code
1.2 Access control	1	AWS + GCP	Cognito ID token; GCP API Gateway JWT check against the user pool's issuer and JWKS; route guard in the app shell	The gateway rejects bad tokens before any function runs. Send the ID token: Cognito access tokens carry no aud claim for the gateway to match

Rubric item	Owner	Cloud	Services	Why, and the alternative
1.3 Permissions and cross-cloud auth	1, all	AWS + GCP	One GCP service account per function; one IAM role per Lambda; Workload Identity Federation (AWS to GCP); OIDC tokens on Pub/Sub push (GCP to AWS)	No long-lived keys anywhere, avoiding the rubric's 0% case of hardcoded credentials
2.1.1 Upload and dedup	2	GCP	Cloud Run function behind API Gateway issuing Cloud Storage V4 signed URLs; Firestore lookup by SHA-256	The browser hashes with Web Crypto; the object is named by its hash and signed with if-generation-match 0, so a duplicate cannot overwrite. Alternative: S3 presigned URLs, which puts storage away from Firestore
2.1.2 Thumbnails and video	2	GCP	Cloud Storage finalize events through Eventarc to Cloud Run functions (Python, OpenCV)	Same cloud as the files, so nothing crosses clouds; give video processing extra memory and a longer timeout
2.1.3 ML tagging	3	GCP	Cloud Run service from a container running YOLO; weights in Cloud Storage; active model path in a Firestore config document	A container holds large model dependencies; editing the config swaps models with no code change. Alternative: Rekognition as the entry model, managed but sends every image across clouds and has fixed labels
2.2.1 Find by tags	4	GCP	Cloud Run function with Firestore map-field queries	Runs next to the data

Rubric item	Owner	Cloud	Services	Why, and the alternative
2.2.2 Thumbnail lookup	4	GCP	Cloud Run function with one Firestore read by file ID	One read, no extra index
2.2.3 Find by uploaded file	3	GCP	The tagging container's inference endpoint, run in memory, then the 2.2.1 function	Reuses the detector; nothing touches storage or the database
2.2.4 Lost pet search	3	GCP	Firestore vector search (`find_nearest`) over embeddings from Vertex AI multimodal embeddings or CLIP in the tagging container	Vector search lives in the database you already have. Vertex is managed but billed per call; CLIP is free but enlarges the container
2.2.5 Chatbot	4	AWS	Amazon Lex V2 bot with a Lambda fulfilment function calling the GCP API Gateway	The spec requires Lex; this is cross-cloud link 2
2.3.1 Bulk tag editing	4	GCP	Cloud Run function with Firestore batched writes	All-or-nothing updates across many files
2.3.2 File deletion	2	GCP	Cloud Run function deleting the original, thumbnail and frames from Cloud Storage, then the Firestore document	Both stores on one cloud, so no cross-cloud step mid-delete
2.3.3 Notifications	1	GCP + AWS	Firestore trigger function publishing tag changes to Pub/Sub; push subscription to an AWS Lambda; SNS topic with an email subscription filter policy per tag	Filter policies give per-species alerts with no code per tag; this is cross-cloud link 3. GCP has no built-in email sender, so SNS stays

Rubric item	Owner	Cloud	Services	Why, and the alternative
3.1 to 3.4 User interface	All	GCP	Firebase Hosting for one single-page app; the Lex widget calls Lex with credentials from a Cognito identity pool	HTTPS by default, which Cognito callback URLs need. Alternative: S3 with CloudFront

At assignment scale most of this sits within free tiers; the per-call costs to watch are Vertex AI embeddings and Lex requests. Set a budget alert on both accounts.



suggested architecture · AWS and GCP, 3 cross-cloud links

Link 1: the gateway checks Cognito tokens. Link 2: the chatbot's Lambda calls the GCP query APIs. Link 3: tag changes reach SNS through Pub/Sub. For the team report (4.2.1), redraw this with the official AWS and GCP icons.